

# **Data Visualisation Assignment**

## **Colour & Encoding Pitfalls in an Ops Dashboard**

Reg No.: 192324066      Name: B.Varun Kumar Reddy

*Tool used for the dashboard: Plotly Dash (Python)*

## Q1. Misuse of Colour (arbitrary rainbow on an ordered variable)

**Dashboard panel:** Delivery performance panel: On-Time Delivery % across the 6 hubs.

### Flawed panel (built first) — the pitfall

The flawed panel colours each hub's bar with an arbitrary qualitative (rainbow-style) palette — colours are assigned by hub identity/alphabetical order, not by the value of On-Time Delivery %. Hue is a categorical channel with no built-in perceptual ordering, so red is not "more" or "less" than blue or green.

### Consequence for the Ops Head

The Ops Head cannot infer which hub is performing best or worst from colour alone. Two adjacent bars — one green (Hub-B, 78%) and one purple (Hub-F, 95%) — give no visual cue about which is higher; the viewer must read every axis label and value to rank hubs, defeating the purpose of a pre-attentive visual summary and slowing decisions in a live ops review.

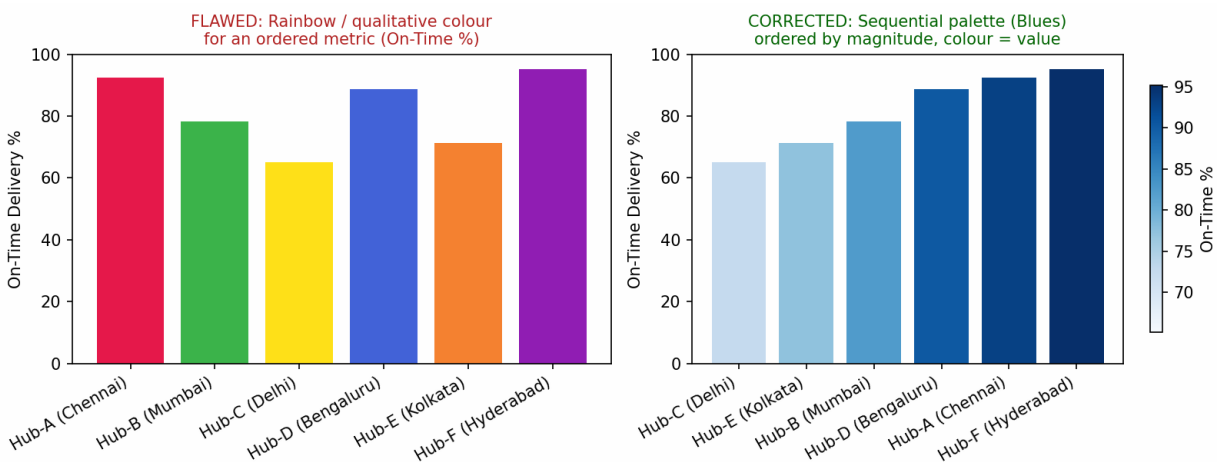


Figure 1. Left: flawed panel. Right: corrected panel (Q1).

### Corrected panel & justification of the fix

The corrected panel (i) sorts hubs by On-Time % and (ii) maps colour through a single-hue sequential scale (Blues), where darker/more saturated blue = higher on-time performance. Because lightness now varies monotonically with the data value, rank order is readable from colour intensity alone, and the sort reinforces it spatially — two redundant but consistent cues rather than one arbitrary and one absent.

**Implemented in:** Plotly Dash (Panel Q1 in the dashboard: 'Misuse of Colour' section)

## Q2. Encoding Too Much / Irrelevant Information

**Dashboard panel:** Hub trade-off panel: delivery time vs cost, exploring which hub needs intervention.

### Flawed panel (built first) — the pitfall

The flawed panel simultaneously encodes 6 variables — x = avg delivery time, y = cost per delivery, colour = hub (categorical), size = delivery volume, marker shape = region, and a text annotation = customer rating.

### Consequence for the Ops Head

Colour-by-hub is redundant: each point is already uniquely identified by its (x, y) position and can be labelled directly, so a 6-colour categorical legend adds a second lookup step with no new information. Marker shape (region) is irrelevant to the stakeholder's actual decision — "which hub is slow/expensive and needs intervention" — because region is not something Ops acts on in this KPI review. With 6 channels active at once the panel forces the viewer to decode a legend, a shape key and a text label simultaneously before any pattern about cost vs. speed can be seen, i.e., cognitive overload that obscures rather than reveals the trade-off.

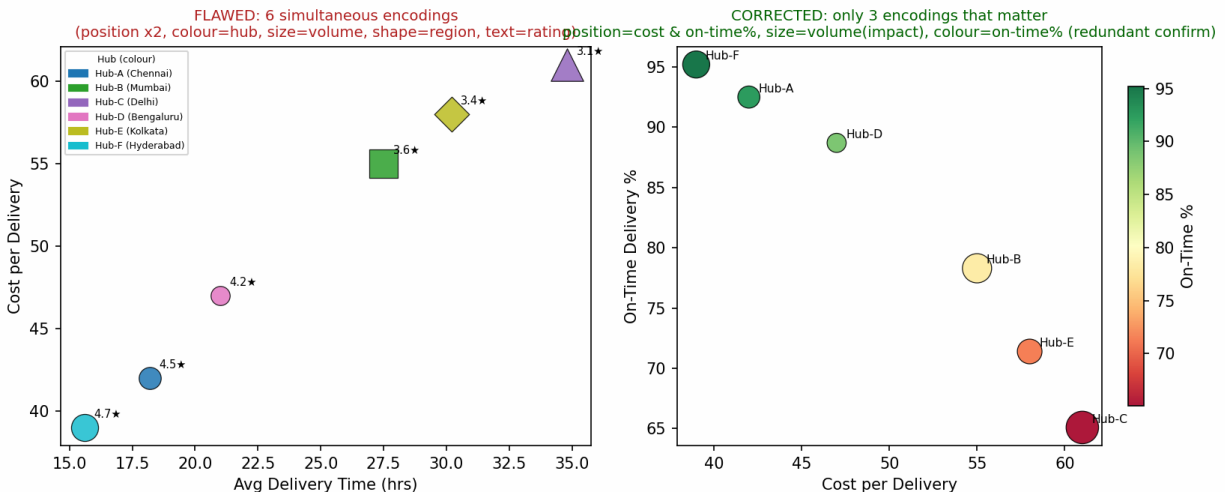


Figure 2. Left: flawed panel. Right: corrected panel (Q2).

### Corrected panel & justification of the fix

The corrected panel keeps only the three encodings that serve the decision: position (cost per delivery on x, On-Time % on y — the two KPIs that define "needs intervention"), size = delivery volume (a hub moving 26,000 parcels matters more than one moving 9,000, so size communicates business impact), and colour = On-Time % on a diverging-friendly sequential scale, which restates the y-axis value redundantly for fast triage. Hub names are shown as plain text labels instead of a colour legend. Region and star-rating are dropped from this panel because they do not change the intervention decision; they remain available as filters elsewhere in the dashboard rather than crowding this view.

**Implemented in:** Plotly Dash (Panel Q2 in the dashboard: 'Over-encoding' section)

### Q3. Non-monotonic Colour Scale ('jet') on a Continuous Metric

**Dashboard panel:** Average delay-per-hub heatmap across 12 months (continuous variable: hours of delay).

#### Flawed panel (built first) — the pitfall

The flawed panel renders average delay (a continuous metric, roughly 1.5 to 9 hours across hubs/months) using the classic 'jet' (rainbow) colormap.

#### Consequence for the Ops Head

Jet is not perceptually monotonic: its lightness oscillates rather than increasing steadily with value. In this dataset, the transition through jet's yellow-to-green band, which falls around 3.5-4.5 hrs of delay, creates a hard visual edge even though the underlying delay values are changing smoothly there — Hub-A and Hub-F cells in that band appear to "jump" in colour when the real change from month to month is gradual (a false boundary). Conversely, in the 6-7 hr range (Hub-D and Hub-E, the worst-performing hubs), jet's red band is nearly flat in perceived brightness, so a genuine and operationally important escalation in delay from 6 hrs to 7+ hrs is visually compressed and easy to miss (a hidden real transition).

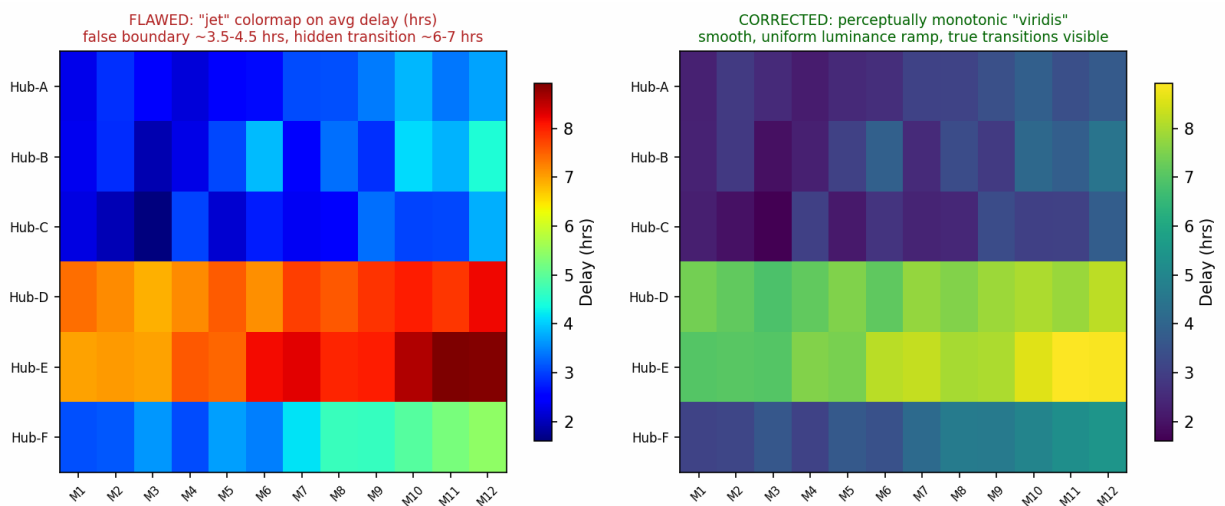


Figure 3. Left: flawed panel. Right: corrected panel (Q3).

#### Corrected panel & justification of the fix

The corrected panel uses Viridis, a perceptually uniform, monotonically-increasing-in-lightness colormap. Equal steps in delay now correspond to equal, consistent steps in perceived colour across the entire range, so the 3.5-4.5 hr region no longer shows a false edge and the genuine escalation around 6-7 hrs (Hub-D, Hub-E) is clearly visible as a continuing gradient. Viridis is also colour-vision-deficiency friendly and prints correctly in greyscale, which further supports Q4's accessibility goal.

**Implemented in:** Plotly Dash (Panel Q3 in the dashboard: 'Non-monotonic Colour Scale' section)

## Q4. Designing for Accessibility and Colour-Vision Deficiency

**Dashboard panel:** Production-line quality panel (5 lines): Pass/Fail status vs. defect-rate threshold.

### Flawed panel (built first) — the pitfall

The flawed panel distinguishes Pass vs. Fail using red bars for Fail and green bars for Pass, with no other visual cue.

### Consequence for the Ops Head

Roughly 8% of men and 0.5% of women have some form of red-green colour-vision deficiency (deuteranopia or protanopia, the most common types). To these viewers, red and green of similar lightness/saturation look like nearly the same muddy yellow-brown, so Line-2 (Fail, red) and Line-1 (Pass, green) become visually indistinguishable — the Plant Manager (or any colour-blind stakeholder on the floor) cannot reliably tell which lines failed quality checks from the panel alone, risking a missed intervention on a failing line.

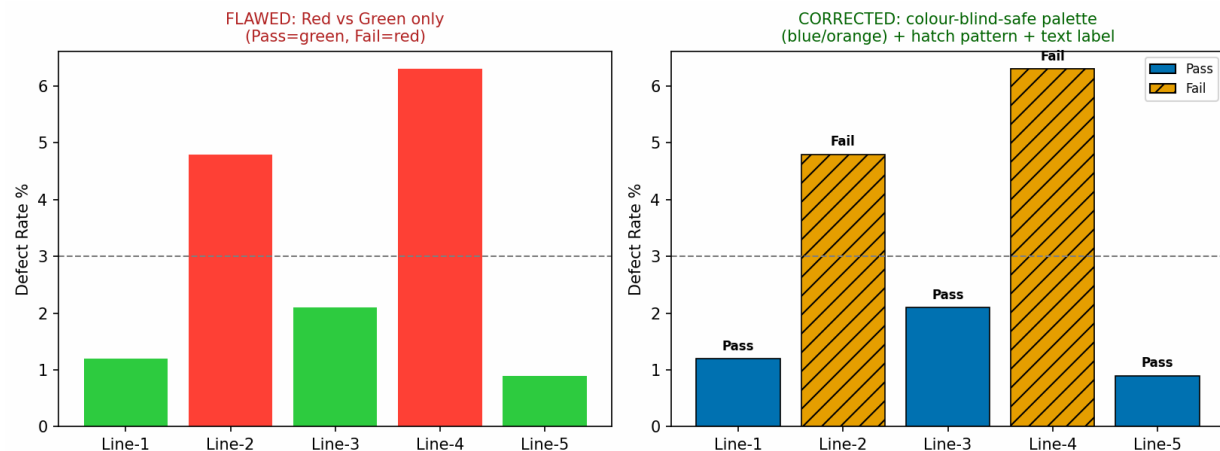


Figure 4. Left: flawed panel. Right: corrected panel (Q4).

### Corrected panel & justification of the fix

The corrected panel switches to the Okabe-Ito colour-blind-safe pair — blue (#0072B2) for Pass and orange (#E69F00) for Fail — which remains distinguishable under deuteranopia, protanopia and tritanopia. It also adds a redundant non-colour cue: a hatch/pattern fill (solid = Pass, diagonal hatch = Fail) plus an explicit text label above each bar. With colour, pattern and text all encoding the same status, the panel remains fully legible even if printed in greyscale or viewed with any form of colour blindness.

**Implemented in:** Plotly Dash (Panel Q4 in the dashboard: 'Accessibility / CVD' section)